

Proposal: NineToothed 算子开发 Skill

一、选手信息

- 姓名: 王一鸣
- 报名赛题编号: T3-1-1
- 提交日期: 2026-05-18

二、.skill 概述

2.1 名称

`ninetoothed-skill` — NineToothed GPU 算子开发完整指导 Skill

2.2 目标用户

使用 Claude Code（或兼容 AI 智能体）完成 NineToothed 生态算子开发的开发者与竞赛参与者。AI 智能体在安装本 .skill 后，能够从一段 CPU 参考实现出发，自主完成分析、代码生成、编译调优、精度验证、性能优化、错误诊断与修复的完整闭环。

2.3 目标任务

覆盖从算子需求到可运行、可测试、性能无明显落后的 GPU 算子实现的完整生命周期，具体包括：

1. **算子开发**：从 CPU 参考实现（NumPy/PyTorch）生成符合 NineToothed 仓库风格的 arrangement + application + premake + torch 接口层 + 测试文件
2. **精度验证**：自动生成 correctness test，覆盖多种 dtype、多种规模、边界情况，执行四项必检（allclose、NaN、Inf、整数精确匹配）
3. **性能优化**：对性能敏感算子提供 benchmark 设计、generated source 分析、性能回退定位和优化建议
4. **错误诊断与修复**：在编译失败、精度不通过、性能异常时，自动定位根因、给出最小修复并验证闭环
5. **集成与风格**：确保生成代码符合仓库命名规范、import 规则、目录结构和测试风格

三、覆盖范围

3.1 算子类型

本 .skill 覆盖 NineToothed 支持的全部 9 种 Arrangement 模式：

模式	名称	复杂度	典型算子
1	Element-wise	低	relu, silu, add, mul, sigmoid, gelu, neg, abs, leaky_relu, meshgrid
2	Reduction	中	softmax, rms_norm, layer_norm, log_softmax
3	Matmul	高	mm
4	Batched Matmul	高	bmm
5	Addmm (组合 mm)	高	addmm
6	Conv2D (im2col + mm)	很高	conv2d
7	Attention	很高	scaled_dot_product_attention
8	Pooling	中	max_pool2d, avg_pool2d
9	RoPE	中	rotary_position_embedding

对于模式 1-3 (element-wise、binary、reduction)，提供经过验证的完整代码模板和自测案例。对于模式 4-9，提供代码骨架和关键规则，AI 智能体可基于模板和仓库现有实现进行开发。

3.2 输入布局

- **连续张量**：标准 contiguous 输入
- **非连续输入**：转置张量（`tensor.T`）、步幅切片（`tensor[:, :2]`）
- **广播场景**：标量与 tensor 运算、不同 ndim tensor 间的广播
- **1D→2D 广播**：如 meshgrid 等需要自定义 arrangement 的场景

已验证行为：NineToothed 自动处理 stride 信息，element-wise、binary、reduction 算子均可直接接受转置张量和步幅切片，无需 `.contiguous()` 转换。

3.3 性能验证方式

1. **Benchmark 对比**：ntops 算子 vs PyTorch 同功能 API，使用 CUDA events 计时（10 warmup + 100 repeat）
2. **Generated Source 检查**：检查 `ninetoothed.make()` 生成的 Triton IR，验证 tile 映射、数据类型、内存访问模式
3. **性能回退定位**：当性能比率 > 1.5x 时，按 6 步诊断流程定位原因（block_size → 内存访问 → 冗余 load/store → 广播开销 → tile 配置 → contiguous/stride）
4. **非连续输入开销分析**：实测对比连续 vs 非连续输入的性能差异

3.4 调试场景

覆盖 12 类常见错误及其诊断修复路径：

错误类型	典型症状	修复方向
Import 错误	ModuleNotFoundError	改用 ntl 等价函数
编译错误	Triton 编译失败	检查 tile/expand/squeeze 链
NaN	isnan 断言失败	加 epsilon, ntl.where 保护
Inf	isinf 断言失败	转 float32 计算
全部错误值	allclose 差异大	检查 arrangement 映射
边界错误	只有边界值错	设置正确的 other 参数
NameError (编译时)	'xxx' is not defined	改用 constexpr Tensor 传入
类型不兼容	IncompatibleTypeError	标量系数改用 constexpr
参数数量不匹配	takes N but M given	检查 premake Tensor 数量

四、包结构与资料

4.1 目录结构

```
ninetoothed-skill/
├─ SKILL.md                # 主入口：触发条件、工作流、约束、验证方式
├─ README.md              # 概述、安装、使用说明
├─ references/
│   ├─ code_templates.md  # 9 种模式的完整代码模板
│   ├─ ntl_api.md         # ninetoothed.language API 参考
│   ├─ tensor_guide.md    # Tensor 声明与元操作参考
│   └─ pitfalls.md        # 14 种常见陷阱及修复方法
├─ examples/
│   ├─ 01_elementwise_leaky_relu.md # 逐元素算子完整示例（含 5 次迭代修复）
│   └─ 02_reduction_log_softmax.md  # 归约算子完整示例（数值稳定）
├─ scripts/
│   ├─ run_tests.py       # 自测任务运行脚本
│   ├─ run_benchmark.py   # Benchmark 运行脚本
│   └─ setup_env.sh       # 环境配置说明脚本
└─ tests/
    ├─ test_leaky_relu.py  # 自测 1: 逐元素算子（含 scalar 参数）
    ├─ test_log_softmax.py # 自测 2: 归约算子（数值稳定）
    ├─ test_non_contiguous.py # 自测 3: 非连续输入场景
    └─ test_benchmark.py   # 自测 4: 性能对比与回退分析
```

4.2 核心资料说明

资料	内容	用途
<code>code_templates.md</code>	9 种 Arrangement 模式的 kernel + torch 完整骨架代码	AI 智能体根据算子分类选择模板，填充计算逻辑
<code>ntl_api.md</code>	ntl 全部操作（数学、规约、比较、类型转换、线性代数、内存、程序信息）	AI 智能体查阅可用的 DSL 函数和签名
<code>tensor_guide.md</code>	Tensor 构造、标量参数选择决策树、元操作链模式	AI 智能体正确声明 Tensor 和选择参数传递方式
<code>pitfalls.md</code>	14 种已知陷阱 + 错误诊断流程图	AI 智能体在遇到错误时快速定位根因
<code>01_elementwise_leaky_relu.md</code>	5 次迭代修复的完整历史	展示闭包错误 → fp64 不兼容 → constexpr 成功的调试路径
<code>02_reduction_log_softmax.md</code>	数值稳定实现在一次迭代中通过	展示 online softmax + float16 精度提升的正确做法

4.3 脚本说明

脚本	用途	运行方式
<code>run_tests.py</code>	顺序运行全部自测任务，报告通过/失败	<code>python scripts/run_tests.py</code>
<code>run_benchmark.py</code>	运行性能对比 benchmark	<code>python scripts/run_benchmark.py</code>
<code>setup_env.sh</code>	检查 Python 版本、配置路径、安装依赖	<code>bash scripts/setup_env.sh</code>

五、核心 workflow

本 .skill 采用六阶段闭环 workflow：

① 分析 CPU 实现 → ② 生成初始算子 → ③ 自动调优 → ④ 精度验证 → ⑤ 性能优化 → ⑥ 输出报告

阶段 1：分析 CPU 实现

- 1. 读取用户提供的 CPU 参考实现
- 2. 使用九种模式分类决策树判断算子类型
- 3. 提取关键信息（维度、dtype、特殊参数、边界情况）
- 4. 确定复用已有 arrangement 还是需要新的

阶段 2：生成初始算子

根据分类选择对应代码模板，生成 4 个文件：

- `kernels/{op_name}.py` — arrangement + application + premake
- `torch/{op_name}.py` — PyTorch 接口层
- 更新 `kernels/__init__.py` 和 `torch/__init__.py`

阶段 3：自动调优

`ninetoothed.make()` 内置 auto-tune，`block_size` 自动搜索最优值。

阶段 4：精度验证

生成测试文件并运行 pytest，覆盖：

- float32 / float16 两种 dtype
- 小 / 中 / 大三种规模
- 边界情况（不整除 `block_size`、极端值、全正/全负）
- 四项必检：allclose + NaN + Inf + 整数精确匹配

阶段 5：性能优化

按优先级评估六项策略：

1. 内存访问模式优化
2. 算子融合
3. 循环展开
4. 减少同步开销
5. 精度策略调整
6. 计算重组

每项策略必须显式标注“已评估”或“不适用（附原因）”。

阶段 6：输出报告

包含算子信息、精度验证结果、性能评估、边界情况列表、迭代历史。

错误恢复与终止保证

- 连续 3 次精度验证不通过 → 停止并报告
- 连续 3 次性能改进 < 5% → 接受当前最优
- 总迭代次数达到 10 次 → 输出当前结果
- 15 项静态验证清单在运行测试前自动检查

六、自测任务设计

自测任务 1：逐元素算子 — `leaky_relu`（含 `scalar` 参数）

项目	内容
输入任务	<code>leaky_relu(x, negative_slope=0.01)</code> — 带可配置斜率的逐元素激活函数
算子类型	Element-wise + 标量参数 (constexpr)
测试覆盖	float32/float16、多种 slope 值 (0.01~2.0)、边界情况 (全正/全负/全零)、3D 输入
验证命令	<code>python -m pytest tests/test_leaky_relu.py -v</code>
迭代历史	3 次迭代: 闭包 NameError → fp64 IncompatibleTypeError → constexpr 成功
性能数据	4096×4096 float32: ntops 0.25ms vs PyTorch 0.23ms (比率 0.92x)
核心验证点	标量参数必须通过 <code>Tensor(0, constexpr=True, value=...)</code> 传入, 禁止闭包

自测任务 2: 归约算子 — log_softmax (数值稳定)

项目	内容
输入任务	<code>log_softmax(x, dim)</code> — 带 max 归一化的数值稳定 log_softmax
算子类型	Reduction (online softmax + log-sum-exp)
测试覆盖	float32/float16、dim=0/dim=-1、3D 输入、数值稳定性 (极端值 -10000~10000)、非整除 block_size
验证命令	<code>python -m pytest tests/test_log_softmax.py -v</code>
迭代历史	1 次通过 (online max + exp 累加算法)
性能数据	4096×1024 float32: ntops 0.36ms vs PyTorch 0.29ms (比率 0.81x)
核心验证点	float16 中间计算必须提升到 float32, padding 值 <code>other=float("-inf")</code>

自测任务 3: 非连续输入场景验证

项目	内容
输入任务	验证多种算子对非连续输入 (转置、步幅切片) 的正确性
算子类型	跨类型验证 (relu、silu、add、leaky_relu、softmax)
测试覆盖	转置张量 (<code>x.T</code>)、步幅切片 (<code>x[:, :2]</code>)、验证输入确实非连续
验证命令	<code>python -m pytest tests/test_non_contiguous.py -v</code>
核心验证点	NineToothed 自动处理 stride 信息, 无需 <code>.contiguous()</code>

自测任务 4：性能对比与回退分析

项目	内容
输入任务	对 7 种算子进行 ntops vs PyTorch 性能对比，并分析非连续输入开销
算子类型	element-wise (relu、silu、leaky_relu)、binary (add)、reduction (softmax、log_softmax)、matmul (mm)
Benchmark 设计	CUDA events 计时，10 warmup + 100 repeat，3 种规模 (256 ² 、1024 ² 、4096 ²)
验证命令	<code>python scripts/run_benchmark.py</code>
性能结论	element-wise: 0.87~0.93x PyTorch；reduction: 0.71~0.81x；matmul: 0.29x (Windows Triton 限制)
核心验证点	性能比率 > 1.5x 时需分析根因，matmul 在 Windows 下受限是已知问题

自测任务汇总

#	算子	类型	是否含 Benchmark	状态
1	leaky_relu	Element-wise + scalar	是	全部通过
2	log_softmax	Reduction	是	全部通过
3	非连续输入	跨类型验证	否	全部通过
4	性能对比	Benchmark + 回退分析	是	已完成

七、Benchmark 设计与预期量化指标

7.1 Benchmark 方法论

- **基线**：PyTorch 同功能 API (`torch.nn.functional.*`)
- **计时方式**：CUDA events (`torch.cuda.Event`)，排除 host-side overhead
- **预热**：10 次预热迭代
- **测量**：100 次重复取中位数
- **同步**：`torch.cuda.synchronize()` 确保计时准确

7.2 输入规模

规模	Shape	说明
小	256×256	验证小规模正确性
中	1024×1024	典型推理场景
大	4096×4096	压力测试，暴露性能瓶颈

7.3 已观测性能指标

算子类型	代表算子	规模	ntops (ms)	PyTorch (ms)	比率
Element-wise	relu	4096 ²	0.22	0.21	0.90x
Element-wise	silu	4096 ²	0.25	0.24	0.93x
Element-wise	leaky_relu	4096 ²	0.25	0.23	0.92x
Binary	add	4096 ²	0.22	0.21	0.87x
Reduction	softmax	4096 ²	0.32	0.26	0.81x
Reduction	log_softmax	4096×1024	0.36	0.29	0.81x
Matmul	mm	1024 ²	1.24	0.36	0.29x

7.4 预期量化目标

指标	目标	依据
Element-wise 性能比率	≥ 0.85x PyTorch	已验证可达 0.87~0.93x
Reduction 性能比率	≥ 0.70x PyTorch	已验证可达 0.71~0.81x
精度测试通过率	100%	四项必检全部通过
迭代修复成功率	≤ 3 次迭代	14 种已知陷阱 + 错误恢复策略
非连续输入正确性	100%	已验证 stride 自动处理

八、风险与边界

8.1 不覆盖的场景

场景	原因	建议
动态 shape (运行时变化 ndim)	NineToothed premake 需固定 ndim	premake 中固定 ndim 参数
递归算法	GPU kernel 禁止递归	改用迭代实现
数据依赖循环	Triton 不支持	改用 range 固定循环
大规模 matmul (Windows)	Windows Triton 优化不足	Linux 环境下性能更好
自定义 CUDA kernel	超出 NineToothed DSL 范围	直接使用 Triton/CUDA
多 GPU / 分布式	NineToothed 当前为单 GPU	需扩展框架能力

8.2 已知限制

- Windows 性能**: Triton 在 Windows 上的优化不如 Linux, matmul 类算子性能显著落后 (0.29x vs Linux 预期 >0.8x)
- float16 精度**: exp/log/sin/cos/tanh 等函数在 float16 下易溢出, 必须先转 float32, 已通过 pitfalls.md 和 code_templates.md 明确标注
- 标量参数传递**: 禁止使用 `Tensor(0, dtype=ninetoothed.float64)` 传浮点系数 (生成 fp64 指针与 fp32 不兼容), 必须使用 `Tensor(0, constexpr=True, value=...)`
- 闭包限制**: NineToothed 通过源码检查编译 application, 闭包变量在 triton 编译时不可见

8.3 依赖与兼容性

依赖	版本要求	用途
Python	≥ 3.10	基础运行时
PyTorch	≥ 2.0	torch 接口层、测试基准
Triton	≥ 3.0	NineToothed 底层编译
NineToothed	主分支	DSL 框架
ntops	主分支	算子库 (开发目标)

8.4 安全约束

- 无 API key、账号凭据或未授权数据
- 无联网依赖 (所有参考资料本地存储)
- 无隐藏评测答案或硬编码任务名
- 无针对评测脚本的规避逻辑
- 所有生成代码可审计、可复现

九、预期效果与量化目标

9.1 对 AI 智能体的改进

维度	无 .skill 基线	有 .skill 预期	改进机制
分类准确率	~60% (常误判模式)	>95%	九种模式决策树 + 模板匹配
首次通过率	~30% (常见闭包、dtype 错误)	>80%	14 种已知陷阱 + 静态验证清单
迭代次数	5-10 次	≤3 次	错误恢复策略表 + 已知陷阱索引
仓库风格一致性	低 (import 错误、命名不规范)	高 (模板强制风格)	代码模板 + 允许/禁止 import 列表
性能意识	无 (不跑 benchmark)	有 (自动 benchmark + 6 策略评估)	benchmark 设计规范 + 性能回退定位流程

9.2 关键差异化设计

1. **分类决策树**：将模糊的"这是什么类型的算子"转化为确定性判断流程，避免 AI 智能体在 9 种模式间犹豫
2. **14 种已知陷阱**：从实际开发中提炼的失败模式索引，AI 智能体遇到错误时可快速匹配并修复
3. **标量参数传递规则**：明确区分 constexpr vs 同类型 Tensor vs 禁止的 fp64 Tensor，避免 3 类常见类型错误
4. **六阶段闭环 + 终止保证**：防止 AI 智能体无限迭代，确保在有限步数内输出可交付结果
5. **性能意识流程化**：将 benchmark 设计、generated source 分析、回退定位从"可选项"变为"必选项"

十、后续计划

10.1 初赛阶段（截至 2026/06/08）

- 完善现有 4 个自测任务的运行记录和结果截图
- 补充至少 2 个额外示例（如 binary 算子 add、广播算子 meshgrid）
- 验证 .skill 在干净仓库环境下的端到端可复现性
- 编写中期报告

10.2 决赛阶段（截至 2026/07/13）

- 根据隐藏任务反馈优化决策树和陷阱索引
- 补充 Attention、Conv2D 等高复杂度模式的示例和模板
- 完善 benchmark 自动化脚本，支持多规模、多 dtype 的批量对比
- 编写最终报告，包含 AI 智能体使用 .skill 前后的对比实验
- 补充 AOT build 配置说明和更多性能优化案例

十一、总结

本 .skill 基于 NineToothed 算子开发的实际经验构建，核心设计原则是"CPU 实现是唯一的精度基准，代码不会说谎"。通过九种模式分类决策树、经过验证的代码模板、14 种已知陷阱索引、六阶段闭环工作流和终止保证，使 AI 智能体在面对九齿算子开发任务时能够：

1. **准确分类**（决策树 → 模板选择）
2. **正确生成**（模板填充 + 静态验证清单）
3. **高效调试**（错误恢复策略 + 陷阱索引）
4. **验证闭环**（四项必检 + benchmark + 报告）
5. **有界迭代**（终止条件 + 最大迭代次数）

自测任务覆盖了竞赛要求的全部 4 类场景（逐元素、归约、非连续输入、性能分析），均已完成并通过验证。